Software Requirements Specification (SRS) v1

Lab 2: Problem Definition + SRS

BusGo

Microservices Bus Ticket Booking Platform

Book your journey across the country, seamlessly.

Team: T07

Version: 1.0

Status: *Draft*

Date: 28 June 2026

Prepared in accordance with the Lab 2 SRS Template.

Contents

1 Document Control

1.1 Metadata

Field	Value
Team	T07
Project Name	BusGo — Microservices Bus Ticket Booking Platform
Repository URL	https://github.com/TAUSEEF-01/Jaabo
Version	1.0
Status	Draft
Date	28 June 2026

1.2 Authors and Reviewers

Role	Name	ID / Email	Date
Author	Md. Tauseef-Ur-Rahman	Roll 24	18-04-2026
Author	Farzana Tasnim	Roll 14	18-04-2026
Team Reviewer	Amina Islam	Roll 36	18-04-2026
Team Reviewer	Tamzid Bin Tariq	Roll 48	18-04-2026

1.3 Revision History

Version	Date	Author	Changes
0.1	20-06-2026	MD. Tauseef	Initial draft: problem definition and scope.
0.2	24-06-2026	Team T07	Added functional requirements, user stories, and data model.
0.3	26-06-2026	Team T07	Added NFRs, risk register, and manual test plan.
1.0	28-06-2026	Team T07	Consolidated review feedback; baseline release.

2 Problem Definition

2.1 Problem Statement

What problem are we solving? Booking intercity bus tickets in many regional markets is still fragmented and unreliable. Travellers must visit physical counters or juggle several operator-specific

websites, none of which expose a unified, real-time view of seat availability, fares, boarding points, or refund policies. Bus operators, in turn, lack a single digital channel to publish trips, manage inventory, run promotions, and reconcile payments. The result is overbooking, double-booked seats, opaque cancellation rules, and a poor customer experience.

BusGo solves this by providing a single, cloud-native, microservices-based platform that unifies trip search, live seat selection, secure payment, e-ticketing, cancellations and refunds, operator self-service, promotions, and administrative oversight behind one consistent web experience and one API gateway.

Why does it matter?

- **For travellers:** a transparent, 24/7 self-service channel with real-time seat locking that eliminates double-booking and surprise cancellation charges.
- **For operators:** a low-cost digital storefront to reach customers, fill seats, and manage routes without building their own software.
- **For the business:** a scalable, independently deployable architecture (12 services behind a Kong gateway) that can grow per-feature without monolithic rewrites.

2.2 Target Users

User Group	Description	Main Needs
Passengers (Customers)	End travellers searching, booking, and cancelling bus tickets via the web app.	Fast search, live seat map, secure payment, instant e-ticket, easy cancellation and refund tracking.
Bus Operators	Companies that own buses, define routes, schedule trips, and run promotions.	Manage buses/routes/trips, view bookings, run deals, monitor analytics and revenue.
Platform Administrators	Internal staff who govern the platform, onboard operators, and resolve disputes.	Operator approval, global oversight, broadcast notifications, audit trails.
Support / Operations Staff	Internal users handling cancellation reviews and customer issues.	Inspect bookings/payments, approve refunds, read audit logs.

2.3 Goals and Non-Goals

Goals

- Allow a passenger to search trips between two cities for a date and book a seat end-to-end.
- Guarantee no double-booking through time-bound seat locking in the Inventory service.
- Issue a verifiable QR e-ticket immediately after successful payment.
- Support cancellations with policy-driven refund calculation.
- Provide an operator portal for managing buses, routes, trips, and promo codes.
- Provide an admin portal with platform-wide oversight and an immutable audit log.
- Deliver the system as independently deployable services behind a single API gateway.

Non-Goals

- Real money settlement: payment is handled via a *mock* gateway, not a live PSP.
- Native mobile applications (iOS/Android) are out of scope for v1.
- Dynamic/surge pricing and AI-based seat recommendations.
- GPS live bus tracking and in-trip telemetry.
- Multi-language / multi-currency localisation beyond a single default locale.

3 Scope and Context

3.1 In Scope

- User registration and authentication (phone + password, OTP verification, JWT, OAuth hook).
- City and trip search with availability filtering (`/api/search`).
- Live seat mapping, locking, and booking lifecycle (Inventory + Booking services).
- Mock payment processing and refund initiation (Payment service).
- Ticket generation with QR codes and downloadable PDF (Ticket service).
- Cancellation requests with refund calculation (Cancellation service).
- Operator self-service portal: buses, routes, trips, bookings, deals, analytics.
- Promo code / discount validation (Deals service).
- Admin portal and broadcast notifications (Admin + Notification services).
- System-wide action logging (Audit service).
- Event-driven cross-service workflows over Kafka; caching/sessions over Redis.

3.2 Out of Scope

- Integration with real payment service providers and bank settlement (handled by mock Bank service).
- Loyalty/rewards programs and wallet balances.
- Third-party aggregator/reseller APIs and white-label embedding.
- Offline counter sales and POS hardware integration.
- Advanced fraud detection and KYC verification workflows.

3.3 Assumptions and Constraints

Technical constraints

- Backend services are FastAPI (Python 3.12+); frontend is React + Vite + TypeScript.
- PostgreSQL 15 with a logical database per service (`auth_db`, `booking_db`, ...).
- Kong API Gateway fronts all services on port 8000; services run on host ports 8101–8112.
- Redis for cache/session; Kafka for asynchronous inter-service events.
- Persistence via SQLAlchemy ORM using `Base.metadata.create_all()` (no destructive migrations).
- Local orchestration via Docker Compose; observability via Prometheus, Grafana, and Loki.

Organizational constraints

- Four-member student team; work delivered across timed lab milestones.
- No production cloud budget; deployment targets local/dev infrastructure.

Timeline constraints

- Lab 2 deliverable (this SRS) due 28 June 2026.
- Manual test plan feeds Lab 3 peer review.

4 Stakeholders and Roles

4.1 Stakeholders

Stakeholder	Interest	Priority
Passenger	Reliable booking, fair refunds, transparent fares.	High
Bus Operator	Higher seat occupancy, easy trip management, timely payouts.	High
Platform Owner (Business)	Transaction volume, platform reliability, operator growth.	High
Platform Administrator	Governance, dispute resolution, compliance via audit logs.	High
Support/Operations Staff	Efficient cancellation handling and issue resolution.	Medium
Development Team (T07)	Maintainable, testable, independently deployable services.	Medium
Payment/Bank Provider (mock)	Correct transaction and refund records.	Medium

4.2 User Roles and Permissions

Role	Description	Permissions
CUSTOMER	Registered traveller using the public web app.	Search trips; lock/select seats; create bookings; pay; view/download tickets; request cancellations; apply promo codes; manage own profile.
OPERATOR	Verified bus operator staff.	CRUD on own buses/routes/trips; view own bookings; create/manage deals; send notifications to own customers; view own analytics.
ADMIN	Platform administrator.	Onboard/deactivate operators; global read across services; broadcast notifications; view audit logs; override/cancel bookings; manage platform-wide deals.
SUPPORT	Operations staff (subset of admin).	View bookings/payments; review and approve/reject cancellation requests; read audit logs.

5 Functional Requirements

Requirement ID format: `FR-T07-###`. Priority uses MoSCoW: **M**ust, **S**hould, **C**ould, **W**on't (this release). Each requirement is atomic and testable.

Req ID	Feature	Requirement Statement	Pri	Acceptance Criteria
FR-T07-001	Registration	The system shall allow a new user to register with full name, unique phone, email, and password.	M	Account created; duplicate phone rejected with 409; password stored as hash.
FR-T07-002	OTP Verify	The system shall verify a phone number via a time-limited OTP.	M	Valid OTP within expiry verifies account; expired/used OTP rejected.
FR-T07-003	Login (JWT)	The system shall authenticate users with phone + password and issue a JWT access and refresh token.	M	Valid credentials return tokens; invalid return 401; refresh token rotates.
FR-T07-004	City Search	The system shall let users search for origin/destination cities.	M	Partial query returns matching cities; empty query handled gracefully.

Req ID	Feature	Requirement Statement	Pri	Acceptance Criteria
FR-T07-005	Trip Search	The system shall return available trips for a given origin, destination, and journey date.	M	Results list operator, departure, fare, and seats available; no past-date trips.
FR-T07-006	Seat Map	The system shall display a live seat map for a selected trip showing AVAIL-ABLE/LOCKED/BOOKEI	M	Seat statuses reflect inventory in real time.
FR-T07-007	Seat Lock	The system shall lock selected seats for a bounded time window during booking.	M	Locked seats unavailable to others until expiry; lock auto-releases on timeout.
FR-T07-008	Passenger Details	The system shall capture passenger details and boarding/dropping points per booking.	M	Booking rejects missing mandatory passenger fields.
FR-T07-009	Create Booking	The system shall create a booking in INITIATED state with an idempotency key.	M	Duplicate idempotency key returns the original booking, not a new one.
FR-T07-010	Apply Promo	The system shall validate and apply a promo code to compute discount.	S	Valid code reduces fare; invalid/expired code rejected with reason.
FR-T07-011	Payment	The system shall process payment for a booking via the mock gateway.	M	Successful payment moves booking to CONFIRMED; failure leaves seats releasable.
FR-T07-012	Ticket Issue	The system shall issue a QR e-ticket on successful payment.	M	Ticket has unique QR data and downloadable PDF; one ticket per booking.
FR-T07-013	View Bookings	The system shall list a user's bookings with current status.	M	Bookings shown newest-first with status and fare.
FR-T07-014	Cancellation	The system shall allow a user to request cancellation of a confirmed booking.	M	Request recorded as PENDING; refund amount computed per policy.
FR-T07-015	Refund	The system shall initiate a refund for an approved cancellation.	S	Refund record created with estimated days; booking marked CANCELLED.

Req ID	Feature	Requirement Statement	Pri	Acceptance Criteria
FR-T07-016	Operator Bus Mgmt	The system shall let operators create, update, and deactivate buses.	M	Bus persisted with unique registration; deactivated bus excluded from new trips.
FR-T07-017	Operator Route Mgmt	The system shall let operators define routes with boarding/dropping points.	M	Route saved with origin, destination, and distance.
FR-T07-018	Operator Trip Mgmt	The system shall let operators schedule trips with fare and seat capacity.	M	Trip created as SCHEDULED; seat inventory generated for the trip.
FR-T07-019	Operator Deals	The system shall let operators create promo codes for their trips.	S	Deal saved with validity window and discount; visible to eligible customers.
FR-T07-020	Operator Analytics	The system shall show an operator revenue and bookings summary.	C	Dashboard renders totals for the operator's trips.
FR-T07-021	Notifications	The system shall send booking/cancellation notifications to users.	S	Confirmation notification generated on CONFIRMED booking.
FR-T07-022	Admin Operator Mgmt	The system shall let admins onboard and deactivate operators.	M	Operator activation toggled; deactivated operator trips hidden.
FR-T07-023	Admin Broadcast	The system shall let admins broadcast notifications to user segments.	C	Broadcast recorded and delivered to targeted users.
FR-T07-024	Audit Log	The system shall record key actions in an immutable audit log.	S	Each create/update/cancel writes an audit entry with actor and timestamp.
FR-T07-025	Profile Mgmt	The system shall let users view and update their profile.	S	Updated fields persist; phone change re-triggers verification.
FR-T07-026	API Gateway Routing	The system shall route all client traffic through the Kong gateway.	M	Requests reach correct service via `/api/<service>/*` routes.
FR-T07-027	Health Checks	The system shall expose a health endpoint per service.	S	`/health` returns 200 when service and its dependencies are ready.

6 User Stories and Use Cases

6.1 User Stories

Story ID	As a...	I want...	So that...	Acceptance Criteria
US-T07-001	Passenger	to register with my phone number	I can access the platform	Account created and OTP sent.
US-T07-002	Passenger	to verify my phone with an OTP	my account is trusted	Valid OTP marks account verified.
US-T07-003	Passenger	to log in securely	I can access my bookings	Correct credentials return a JWT.
US-T07-004	Passenger	to search trips between two cities on a date	I can plan my journey	Matching trips with fares are listed.
US-T07-005	Passenger	to view a live seat map	I can pick my preferred seat	Seat statuses are accurate.
US-T07-006	Passenger	seats I select to be held briefly	no one else takes them while I pay	Selected seats are locked until expiry.
US-T07-007	Passenger	to enter passenger and boarding details	my ticket is correct	Booking stores passenger data.
US-T07-008	Passenger	to apply a promo code	I pay a lower fare	Valid code reduces total fare.
US-T07-009	Passenger	to pay for my booking	my seat is confirmed	Successful payment confirms booking.
US-T07-010	Passenger	to receive a QR e-ticket	I can board the bus	Ticket with QR is issued and downloadable.
US-T07-011	Passenger	to see all my bookings	I can track my trips	Bookings list shows status.
US-T07-012	Passenger	to cancel a booking	I can get a refund when plans change	Cancellation request is recorded with refund amount.
US-T07-013	Passenger	to track my refund status	I know when money returns	Refund shows status and estimated days.
US-T07-014	Passenger	to manage my profile	my details stay current	Profile updates persist.
US-T07-015	Operator	to add my buses	I can schedule trips on them	Bus saved with unique registration.

Story ID	As a...	I want...	So that...	Acceptance Criteria
US-T07-016	Operator	to define routes with stops	customers see boarding points	Route saved with stops.
US-T07-017	Operator	to schedule trips with fares	customers can book them	Trip created and bookable.
US-T07-018	Operator	to create promo codes	I can attract more bookings	Deal active within its validity window.
US-T07-019	Operator	to view bookings on my trips	I can plan operations	Operator sees own trip bookings.
US-T07-020	Operator	to see revenue analytics	I can track performance	Dashboard shows totals.
US-T07-021	Admin	to approve operators	only legitimate operators sell tickets	Operator activation toggled.
US-T07-022	Admin	to broadcast notifications	I can inform users of changes	Broadcast delivered to targets.
US-T07-023	Admin	to view audit logs	I can investigate issues	Audit entries are searchable.
US-T07-024	Support	to review cancellation requests	refunds are fair and correct	Request approved/rejected with reason.

6.2 Critical Use Cases

UC-1: Book a Bus Ticket (End-to-End)

Actor: Passenger (CUSTOMER).
Preconditions: User is registered, verified, and logged in; at least one matching trip exists.
Main flow:

1. User searches origin, destination, and journey date.
2. System returns available trips; user selects one.
3. System displays the live seat map; user selects available seat(s).
4. Inventory service locks the seats for the configured window.
5. User enters passenger and boarding/dropping details.
6. Booking service creates a booking (INITIATED) with an idempotency key.
7. User optionally applies a valid promo code; fare is recomputed.
8. User pays via the mock Payment service.
9. On success, booking moves to CONFIRMED and the Ticket service issues a QR e-ticket.
10. Notification service sends a booking confirmation.

Alternate / exception flow:

- 4a. Selected seat already locked/booked → system prompts re-selection.
- 7a. Invalid/expired promo → discount rejected with reason; user may retry.
- 8a. Payment fails or seat lock expires → booking not confirmed; seats released.

Postconditions: A CONFIRMED booking, a payment record, and a valid e-ticket exist; seats are BOOKED.

UC-2: Cancel a Booking and Process Refund

Actor: Passenger; reviewed by Support/Admin.
Preconditions: User has a CONFIRMED booking eligible for cancellation.
Main flow:

1. User opens a confirmed booking and requests cancellation with a reason.
2. Cancellation service computes refund amount per policy and records a PENDING request.
3. Support/Admin reviews and approves the request.
4. Payment service initiates a refund with estimated processing days.
5. Booking is marked CANCELLED; seats are released back to inventory.
6. Notification service informs the user.

Alternate / exception flow:

- 3a. Request rejected → status REJECTED with rejection reason; no refund.
- 2a. Booking past cancellation cutoff → refund amount is zero per policy.

Postconditions: Booking CANCELLED; refund record created; seats AVAILABLE again.

UC-3: Operator Schedules a Trip

Actor: Operator (OPERATOR).
Preconditions: Operator is active and has at least one active bus and route.
Main flow:

1. Operator selects a bus and route and sets departure/arrival times, fare, and capacity.
2. Operator service creates the trip in SCHEDULED state.
3. Inventory service generates seat inventory for the trip.
4. Trip becomes searchable to customers.

Alternate / exception flow:

- 1a. Inactive bus/route selected → trip creation rejected.
- 2a. Overlapping trip on the same bus → conflict warning.

Postconditions: A bookable SCHEDULED trip with generated seat inventory exists.

7 Data Requirements

7.1 Core Entities

Entity	Description	Key Fields	Constraints
User	A platform account.	id, phone, email, role, password_hash	phone unique; role ∈ {CUSTOMER, ADMIN, OPERATOR}.
Operator	A bus company.	id, name, license_no, commission_rate	license_no present; commission_rate ≥ 0.
Bus	A physical vehicle.	id, operator_id, registration_no, bus_type, total_seats	registration_no unique; total_seats > 0.
Route	Origin–destination path.	id, operator_id, origin_city, destination_city, distance_km	origin ≠ destination.
Trip	A scheduled journey.	id, bus_id, route_id, departure_datetime, fare_amount, available_seats	status ∈ {SCHEDULED, CANCELLED, COMPLETED}.
SeatInventory	Per-seat state for a trip.	id, trip_id, seat_number, seat_type, status	status ∈ {AVAILABLE, LOCKED, BOOKED}.
Booking	A reservation.	id, user_id, trip_id, seat_numbers, total_fare, status, idempotency_key	idempotency_key unique.
Payment	A payment attempt.	id, booking_id, amount, method, status, gateway_transaction_id	gateway_transaction_id unique.
Refund	A refund for a payment.	id, payment_id, amount, status, estimated_days	amount ≤ payment.amount.
Ticket	An issued e-ticket.	id, booking_id, qr_code_data, pdf_url, status	booking_id unique; qr_code_data unique.
CancellationRequ	A cancellation case.	id, booking_id, user_id, status, refund_amount	status ∈ {PENDING, APPROVED, REJECTED}.

Entity	Description	Key Fields	Constraints
Deal	A promo code.	id, code, discount, validity window	code unique within operator.
AuditLog	Immutable action record.	id, actor, action, entity, timestamp	append-only.

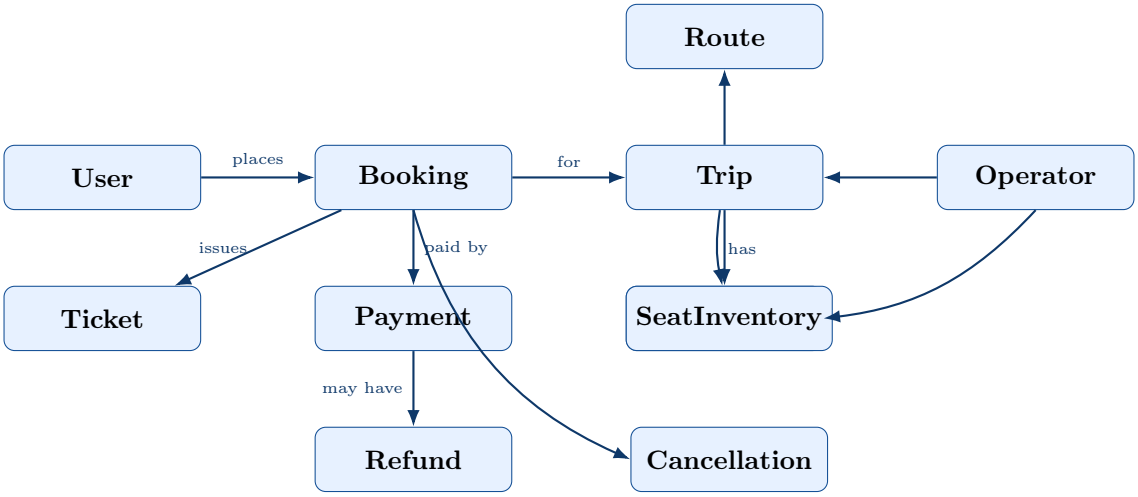
7.2 Data Dictionary

Field	Type	Req?	Validation / Constraint	Example
users.phone	String	Yes	Unique; numeric; 11 digits.	01700000000
users.email	String	No	RFC-5322 email format.	user@busgo.dev
users.role	Enum	Yes	CUSTOMER / ADMIN / OPERATOR.	CUSTOMER
users.password_hasl	String	Yes	Bcrypt hash; never plaintext.	$2b$12$...
otp_records.otp_co	String	Yes	4–6 digits; single-use.	482913
trips.fare_amount	Numeric	Yes	> 0; two decimal places.	850.00
trips.departure_dat	DateTime	Yes	Future timestamp at creation.	2026-07-02T22:00
seat_inventory.statu	Enum	Yes	AVAILABLE / LOCKED / BOOKED.	AVAILABLE
seat_inventory.lock_	DateTime	No	Set when status = LOCKED.	2026-06-28T14:10
bookings.seat_numb	JSONB	Yes	Non-empty array of seat IDs.	["A1","A2"]
bookings.idempoten	String	Yes	Unique per booking attempt.	idem-7f3a...
bookings.status	Enum	Yes	INITIATED / CONFIRMED / CANCELLED / EXPIRED.	CONFIRMED
bookings.total_fare	Numeric	Yes	≥ 0 after discount.	765.00
payments.method	Enum	Yes	Supported mock method.	CARD
payments.status	Enum	Yes	PENDING / SUCCESS / FAILED.	SUCCESS
refunds.estimated_d	Integer	No	$0 \leq$ days ≤ 14.	5
tickets.qr_code_dat	String	Yes	Unique signed token.	QR-9b21...
cancellation_request	Numeric	Yes	≥ 0 and $\leq$ booking fare.	612.00

7.3 Data Model

The platform follows a *database-per-service* pattern: each microservice owns a logical PostgreSQL database and references entities in other services by UUID rather than by hard foreign keys across

databases. The logical relationships are shown below.



Major relationships

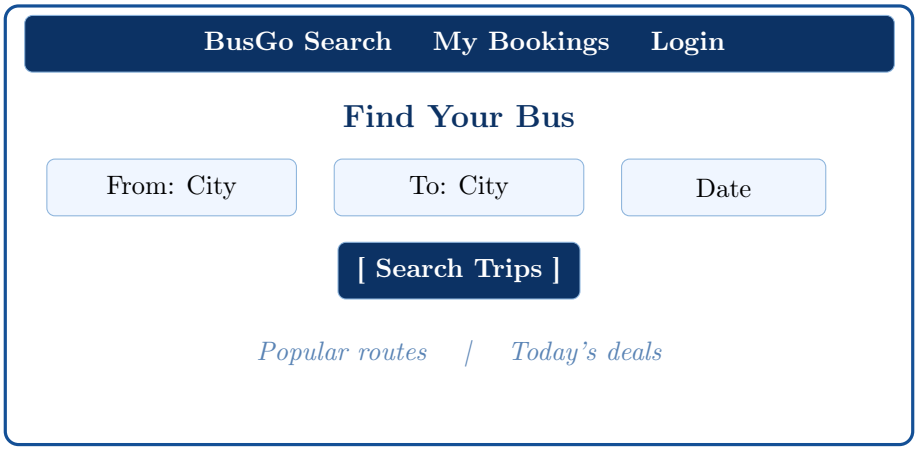
- A **User** places many **Bookings**; each Booking references one **Trip**.
- A **Trip** belongs to one **Operator**, runs on one **Bus** along one **Route**, and owns many **SeatInventory** rows.
- A **Booking** has one **Payment** and (on confirmation) one **Ticket**.
- A **Payment** may have one **Refund**; a **Booking** may have one **CancellationRequest**.

8 UI and UX Requirements

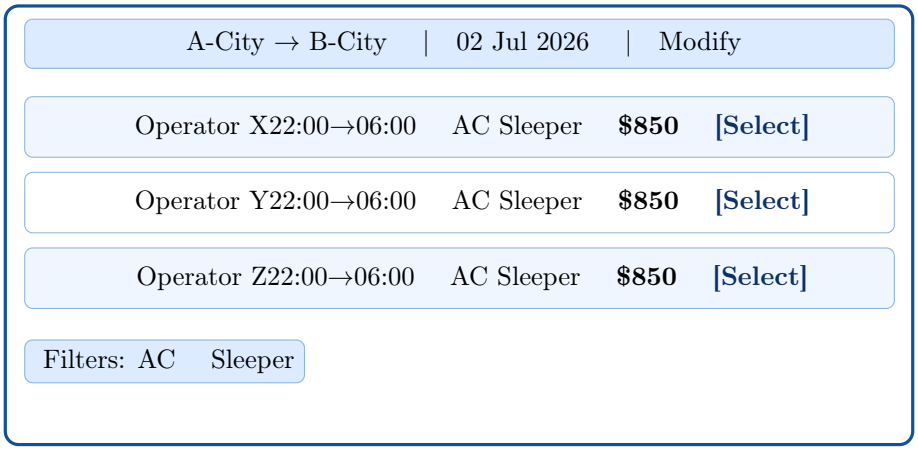
8.1 Wireframes

The following low-fidelity wireframes cover the primary passenger flow plus the operator portal.

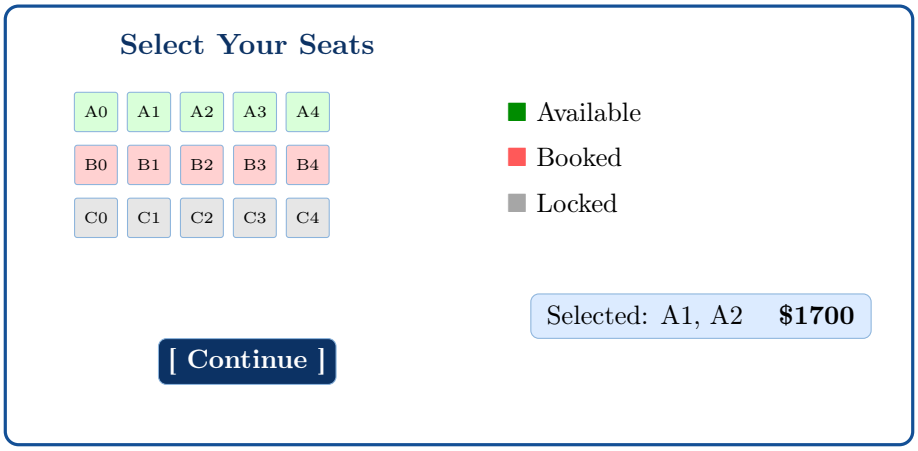
Wireframe 1 — Home / Trip Search



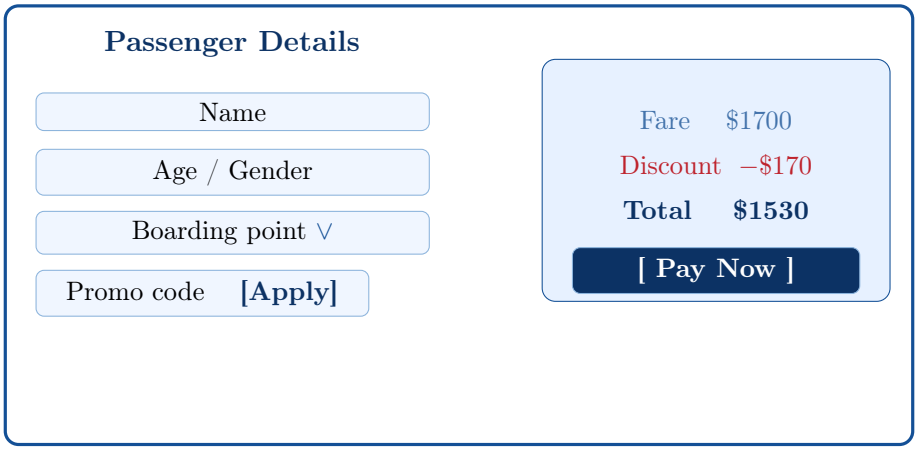
Wireframe 2 — Search Results



Wireframe 3 — Seat Selection

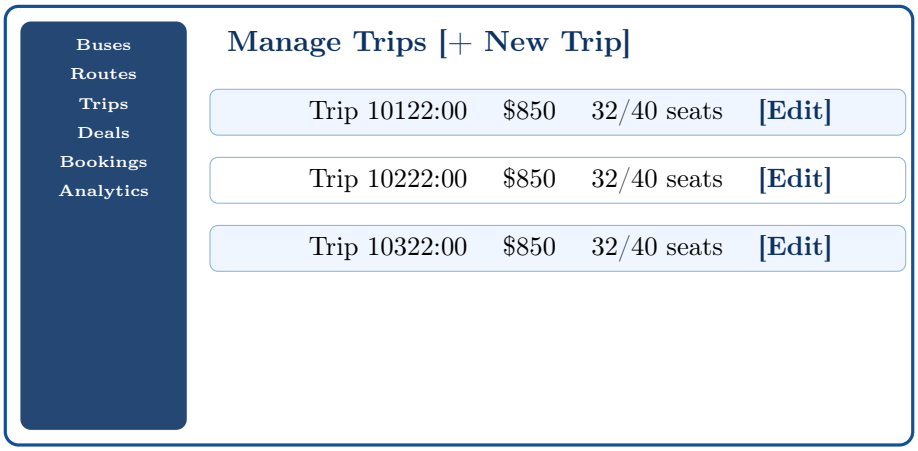


Wireframe 4 — Passenger Details & Payment



Wireframe 5 — Booking Confirmation & E-Ticket

Wireframe 6 — Operator Portal (Manage Trips)



8.2 UX and Accessibility Baseline

- **Clear navigation and labels:** persistent top navigation; descriptive button labels (e.g., "Search Trips", "Pay Now"); breadcrumb of the booking steps.
- **Keyboard accessibility:** all core flows (search, seat select, pay) operable via keyboard; visible focus states; logical tab order.
- **Readable contrast and typography:** WCAG AA contrast ($\geq$ 4.5:1) for text; minimum 16 px body text; colour is never the sole status indicator (seat states also labelled).
- **Responsive behavior:** layouts adapt from desktop to mobile widths; seat map and booking summary reflow on small screens.
- **Feedback:** loading states, inline validation messages, and toast notifications for success/error.

9 Non-Functional Requirements

NFR ID format: `NFR-T07-###`.

NFR ID	Category	Requirement	Target / Metric
NFR-T07-001	Performance	Trip search results shall return quickly under normal load.	p95 latency $\leq$ 800 ms for search at 50 concurrent users.

NFR ID	Category	Requirement	Target / Metric
NFR-T07-002	Performance	Seat map shall load promptly after trip selection.	p95 $\leq$ 1 s.
NFR-T07-003	Scalability	Services shall scale horizontally behind the Kong gateway.	Each service runs $\geq$ 2 replicas with load balancing.
NFR-T07-004	Availability	Core booking path shall remain available during single-service restarts.	$\geq$ 99% monthly availability in dev environment.
NFR-T07-005	Reliability	Seat locks shall auto-expire to prevent permanent blocking.	100% of expired locks released within the lock window + 60 s.
NFR-T07-006	Consistency	Booking creation shall be idempotent.	Duplicate idempotency key never creates a second booking.
NFR-T07-007	Security	Passwords shall be stored only as salted hashes.	0 plaintext passwords; bcrypt with cost $\geq$ 12.
NFR-T07-008	Security	APIs shall require JWT auth for protected routes.	401 returned for missing/invalid tokens.
NFR-T07-009	Security	All client traffic shall pass through the API gateway.	No service is directly reachable by clients in deployment.
NFR-T07-010	Usability	Core booking flow shall be completable in few steps.	$\leq$ 5 screens from search to confirmation.
NFR-T07-011	Accessibility	UI shall meet WCAG 2.1 AA basics.	Contrast $\geq$ 4.5:1; keyboard operable core flows.
NFR-T07-012	Maintainability	Each service shall be independently deployable.	Service can be rebuilt/restarted without redeploying others.
NFR-T07-013	Observability	System shall expose metrics, logs, and health.	Prometheus metrics, Loki logs, `/health` per service.
NFR-T07-014	Portability	Stack shall run via Docker Compose on a dev machine.	`make up` brings up all services.
NFR-T07-015	Data Integrity	Audit log shall be append-only.	No update/delete on audit entries.
NFR-T07-016	Recoverability	Services shall recover from transient Kafka/Redis outages.	Service restarts and reconnects without manual data fixes.

10 Risks and Open Questions

10.1 Risk Register

Risk ID	Description	Probability	Impact	Mitigation
RISK-T07-001	Distributed consistency: seat lock and booking span two services and may diverge.	**Medium**	**High**	Time-bound locks + idempotency keys + reconciliation job.
RISK-T07-002	Kafka unavailability causes services to crash-loop on startup.	**Medium**	**High**	Retry/backoff on connect; health gating; documented recovery steps.
RISK-T07-003	Schema changes require volume wipe (`create_all`, no migrations).	**High**	**Medium**	Introduce Alembic migrations before production; seed scripts for reload.
RISK-T07-004	Mock payment behaviour diverges from a real PSP.	**High**	**Medium**	Encapsulate gateway behind an interface for later swap.
RISK-T07-005	Gateway misrouting (singular vs plural paths) breaks endpoints.	**Medium**	**Medium**	Kong config supports both forms; route tests in CI.
RISK-T07-006	Port conflicts / Windows excluded-port bind failures in dev.	**Medium**	**Low**	Documented 850x port convention; configurable host ports.
RISK-T07-007	Team capacity: four-member student team across timed labs.	**Medium**	**Medium**	Prioritise Must requirements; defer Could/Won't items.
RISK-T07-008	Security: JWT/secret handling in dev may leak.	**Low**	**High**	Environment-based secrets; rotate keys; no secrets in repo.

10.2 Open Questions

- What exact refund policy tiers apply (time-before-departure brackets and percentages)?
- What is the precise seat-lock duration, and should it differ by trip popularity?
- Which mock payment methods must be supported in v1 (card, wallet, netbanking)?
- Should OAuth login be in v1 scope or deferred to a later release?
- What user segmentation is required for admin broadcast notifications?
- Is multi-seat-per-booking limited (e.g., max seats per transaction)?

11 Testability and Traceability

11.1 Requirement Traceability

Requirement ID	Story / Use Case	Planned Test ID
FR-T07-001	US-T07-001	TC-T07-001
FR-T07-002	US-T07-002	TC-T07-002
FR-T07-003	US-T07-003	TC-T07-003
FR-T07-005	US-T07-004 / UC-1	TC-T07-004
FR-T07-006	US-T07-005 / UC-1	TC-T07-005
FR-T07-007	US-T07-006 / UC-1	TC-T07-006
FR-T07-009	US-T07-007 / UC-1	TC-T07-007
FR-T07-010	US-T07-008	TC-T07-008
FR-T07-011	US-T07-009 / UC-1	TC-T07-009
FR-T07-012	US-T07-010 / UC-1	TC-T07-010
FR-T07-013	US-T07-011	TC-T07-011
FR-T07-014	US-T07-012 / UC-2	TC-T07-012
FR-T07-015	US-T07-013 / UC-2	TC-T07-013
FR-T07-016	US-T07-015	TC-T07-014
FR-T07-018	US-T07-017 / UC-3	TC-T07-015
FR-T07-019	US-T07-018	TC-T07-008
FR-T07-022	US-T07-021	TC-T07-014
FR-T07-024	US-T07-023	TC-T07-013
FR-T07-026	UC-1	TC-T07-009

11.2 Initial Manual Test Plan

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T07-001	Register new user	Phone not registered	Submit registration form with valid data	Account created; OTP sent; 201 response.
TC-T07-002	OTP verification	User registered, OTP issued	Enter valid OTP before expiry	Account marked verified.
TC-T07-003	Login success	Verified user exists	Submit correct phone + password	JWT access & refresh tokens returned.

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T07-004	Trip search	Trips seeded for route/date	Search A-City → B-City for a future date	Matching trips with fares listed.
TC-T07-005	Seat map load	A trip selected	Open seat map	Seats render with correct AVAILABLE/LOCKED/BOOKED states.
TC-T07-006	Seat lock holds	Two sessions, same trip	Session A selects A1; Session B tries A1	A1 unavailable to B until lock expiry.
TC-T07-007	Create booking	Seats locked by user	Submit passenger details	Booking created in INITIATED with idempotency key.
TC-T07-008	Apply valid promo	Active promo exists	Enter valid code at payment	Discount applied; total fare reduced.
TC-T07-009	Successful payment	Booking INITIATED	Pay via mock gateway (success)	Booking CONFIRMED; payment SUCCESS.
TC-T07-010	Ticket issued	Booking CONFIRMED	Open confirmation page	Unique QR ticket and PDF available.
TC-T07-011	View my bookings	User has bookings	Open My Bookings	Bookings listed newest-first with status.
TC-T07-012	Request cancellation	Confirmed booking exists	Submit cancellation with reason	PENDING request with computed refund amount.
TC-T07-013	Approve refund	Pending cancellation exists	Support approves request	Refund record created; booking CANCELLED; seats released.
TC-T07-014	Operator creates bus	Operator active	Add bus with unique registration	Bus saved; duplicate registration rejected.
TC-T07-015	Operator schedules trip	Active bus & route exist	Create trip with fare/capacity	Trip SCHEDULED; seat inventory generated.
TC-T07-016	Duplicate registration	Phone already used	Register with same phone	409 conflict; no duplicate account.
TC-T07-017	Invalid login	Verified user exists	Submit wrong password	401; no token issued.
TC-T07-018	Expired promo rejected	Expired promo exists	Enter expired code	Code rejected with reason; no discount.

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T07-019	Payment failure path	Booking INITIATED	Pay via mock gateway (failure)	Booking not confirmed; seats releasable.
TC-T07-020	Gateway routing	Services up via Kong	Call `/api/search/...`	Request routed to Search service (200).

12 Lab 2 Submission Checklist

☒ Problem statement and scope are clear.

☒ Minimum 12 user stories with acceptance criteria (24 provided).

☒ Functional requirements table is complete (27 requirements).

☒ Data model and data dictionary are attached.

☒ Minimum 5 wireframes or flow screens included (6 provided).

☒ NFR section is completed (16 NFRs).

☒ Risks and open questions are documented.

☒ Initial manual test plan (15 cases) is included (20 provided).

End of Document — BusGo SRS v1.0 — Team T07